

Proyecto #2 – Prolog

Almacén Robótico Lógico

En un almacén robótico altamente automatizado, ha ocurrido un error en el sistema de clasificación. Una "Caja Objetivo" de vital importancia ha quedado atrapada entre otras cajas ordinarias.

Su objetivo como especialista en **programación lógica** es diseñar un conjunto de predicados (reglas y hechos) en Prolog que permitan encontrar la secuencia de movimientos mínima necesaria para controlar a un Robot de carga y empujar la Caja Objetivo hasta la zona de extracción.

El programa debe considerar las siguientes reglas y restricciones del entorno:

1. El almacén se representa como una matriz lógica de 6x6 (coordenadas de la (0,0) a la (5,5)). La casilla (0,0) es la esquina superior izquierda.
2. Existen tres tipos de entidades en el tablero: Un Robot, una Caja Objetivo, y múltiples Cajas de Bloqueo (obstáculos). Cada entidad ocupa exactamente una casilla.
3. El Robot puede moverse en cuatro direcciones representadas por átomos: Arriba ('u'), Abajo ('d'), Izquierda ('l') y Derecha ('r').
4. Si el Robot se mueve hacia una casilla vacía, simplemente cambia de posición.
5. Mecánica de Empuje: Si el Robot se mueve hacia una casilla ocupada por una caja (sea la Objetivo o de Bloqueo), el Robot empujará la caja una casilla en la misma dirección, si y solo si la casilla detrás de la caja está vacía y dentro de los límites del tablero.
6. Prohibición de empuje múltiple: El Robot no tiene fuerza para empujar dos cajas al mismo tiempo. Si detrás de la caja que se desea empujar hay otra caja, el movimiento es inválido.
7. Ninguna entidad puede salirse de los límites del tablero.
8. La zona de extracción (la meta) es siempre la coordenada (5,5). El juego termina exitosamente en el instante en que la Caja Objetivo llega a esa coordenada.
9. Se debe minimizar la cantidad de movimientos (mejor solución).

Parte 1 - Inicialización del tablero

La representación de las entidades en el tablero se hará mediante predicados dinámicos: `robot(Row, Col)`, `caja_objetivo(Row, Col)` y `caja_bloqueo(Row, Col)`.

Implemente el predicado `initialBoard(RobotCoord, TargetCoord, BlockingBoxes)` que reciba las coordenadas iniciales, valide que no existan solapamientos ni entidades fuera de los límites de la matriz 6x6 y, si todo es válido, cargue la base de conocimientos con los hechos correspondientes. Si hay algún error lógico o espacial, el predicado debe fallar y no cargar nada en la base de conocimientos.

Ejemplos:

```
1) ?- initialBoard((0,0), (2,2), [(1,1), (3,3)]).  
true.
```

Base de conocimientos:

```
robot(0, 0).  
caja_objetivo(2, 2).  
caja_bloqueo(1, 1).  
caja_bloqueo(3, 3).
```

```
2) ?- initialBoard((1,1), (1,1), [(3,3)]).  
false. (Falla por solapamiento entre el Robot y la Caja Objetivo)
```

Parte 2 – Validación de movimientos

Antes de realizar un movimiento, se debe verificar lógicamente si es posible. Implemente el predicado `isValidMove(CurrentState, Move)` que verifique si desplazar el Robot con el movimiento `Move` es válido en el tablero actual.

Un movimiento es válido si:

1. El Robot (y la caja si es empujada) no se sale de los límites del tablero 6x6.
El trayecto cumple con las reglas de empuje de cajas y prohibición de empuje múltiple.

Considerando que el estado base se representa como una estructura: `state((2,2), (2,3), [(2,4)])` (Robot en (2,2), Caja Objetivo en (2,3) y una Caja de Bloqueo justo detrás en (2,4)).

Ejemplos:

1) Movimiento simple a una casilla vacía (Arriba):

```
?- isValidMove(state((2,2), (2,3), [(2,4)]), 'u').
```

true. (El Robot pasaría de (2,2) a (1,2) sin problema)

2) Empuje inválido (Doble Caja):

```
?- isValidMove(state((2,2), (2,3), [(2,4)]), 'r').
```

false.

(El Robot intenta moverse a la derecha empujando la Caja Objetivo, pero choca con la Caja de Bloqueo en (2,4))

3) Empuje fuera del tablero:

```
?- isValidMove(state((4,4), (5,4), []), 'd').
```

false.

(Al moverse hacia abajo, empujaría la Caja Objetivo a la fila 6, lo cual excede el límite del tablero (0-5))

Parte 3 – Ejecución de movimiento

Implemente la lógica para representar el cambio de estado. Dado que en Prolog la búsqueda de caminos suele requerir generar nuevos estados sin destruir el anterior (para poder hacer backtracking o BFS), implemente el predicado:

```
moveRobot(CurrentState, Move, NewState).
```

Este predicado relaciona un estado del tablero con el siguiente estado resultante de aplicar un movimiento válido. Recuerde que si el Robot se mueve hacia una caja, la coordenada de esa caja también debe actualizarse en la estructura del `NewState`.

Considerando el estado base: `state((2,2), (2,3), [(4,4)])`

Ejemplos:

1) Movimiento simple (Mover abajo):

```
?- moveRobot(state((2,2), (2,3), [(4,4)]), 'd', NewState).
```

NewState = state((3,2), (2,3), [(4,4)]).

2) Empujar Caja Objetivo (Mover derecha):

```
?- moveRobot(state((2,2), (2,3), [(4,4)]), 'r', NewState).
```

NewState = state((2,3), (2,4), [(4,4)]).

(El Robot toma el lugar de la caja, y la caja se desplaza una casilla a la derecha)

3) Empujar Caja de Bloqueo:

```
?- moveRobot(state((4,3), (1,1), [(4,4)]), 'r', NewState).
```

NewState = state((4,4), (1,1), [(4,5)]).

(El Robot empuja la Caja de Bloqueo hacia la derecha.)

Parte 4 – Solución

Implementar el predicado `solveWarehouse(StartState, Solution)` para encontrar la secuencia de movimientos óptima.

Dónde:

- `StartState`: Configuración inicial del estado.
- `Solution`: Lista de movimientos que llevan a la solución.

Debe utilizar un algoritmo de búsqueda (preferiblemente BFS - Búsqueda en Anchura) para garantizar que la solución encontrada sea la más corta en número de movimientos. Evite ciclos (estados repetidos).

Ejemplos:

1) Caso simple (1 movimiento para ganar):

```
?- solveWarehouse(state((5,3), (5,4), []), Solution).
```

Solution = ['r']. (El Robot empuja a la derecha y la Caja Objetivo llega de (5,4) a (5,5))

2) Caso de posicionamiento complejo (3 movimientos):

```
?- solveWarehouse(state((4,4), (4,5), []), Solution).
```

```
Solution = ['u', 'r', 'd'].
```

(El Robot no puede empujar la caja directamente. Debe subir a (3,4), moverse a la derecha a (3,5) para ubicarse arriba de la caja, y empujarla hacia abajo para que alcance (5,5))

3) Caso sin solución (Atrapado):

```
?- solveWarehouse(state((0,0), (0,1), [(0,2), (1,1), (1,2)]), Solution).
```

```
false.
```

(El Robot y la Caja Objetivo están completamente encerrados en la esquina superior izquierda por Cajas de Bloqueo. Al no haber camino posible, el predicado falla)

Puntuación

- Parte 1, 1pto.
- Parte 2 y 3, 2 ptos c/u.
- Parte 4, 14 ptos.
- Readme, 1 pto.

Consideraciones para la entrega

- Debe utilizar SWI-Prolog.
- Realice las validaciones adicionales que considere necesarias.
- Los predicados definidos deben estar dentro de un archivo llamado **Proyecto2.pl**. Puede definir los predicados auxiliares con cualquier nombre que describa su propósito, pero los predicados descritos arriba deben mantener su nombre y aridad.
- Documente la solución (representación, hechos definidos, estrategia utilizada, etc.) en un archivo en formato *markdown* (**README.md**). No están permitidos documentos en ningún otro formato.
- No está permitido utilizar soluciones de terceros (incluyendo soluciones generadas por LLMs). Cualquier material consultado o prompts para ideas de solución o documentación debe ser referenciado en una sección del **README.md**, indicando para qué sirvió.
- El proyecto se puede realizar en grupos de **hasta dos estudiantes de cualquier sección** y la fecha de entrega será el **lunes 22/06/2026**, hasta las 11:59 pm (VET).
- El proyecto se debe adjuntar por e-mail a la dirección `ldpucv@gmail.com` usando como asunto:
"[Proyecto Prolog]" <DatosEstudiante> [; <DatosEstudiante>]
donde <DatosEstudiante> ::= <Apellidos><Nombres>,<CI>.